

aosp-update-engine

AOSP update_engine / Android OTA — Stack Research Note

Field	Value
Revision	1
Created	2026-06-07
Last modified	2026-06-07
Status	active
Status summary	Initial research complete. Native Android A/B + Virtual A/B mechanics, <code>payload.bin</code> generation via <code>ota_from_target_files</code>, and the <code>UpdateEngine.applyPayload</code> client API are documented from primary AOSP sources. Server-side responsibilities for streaming OTA are characterized. Conclusion: native-on-device <code>update_engine</code> is sufficient for the Helix Android 15 client; Helix only needs to build the payload and serve it.
Issues	A few exact constants (Virtual A/B GMS version threshold, compression % figures) are sourced from AOSP docs but should be re-confirmed against the Android 15/16 docs revision before they are quoted in contracts. Marked UNVERIFIED where applicable.
Fixed	N/A (first revision)
Continuation	Follow-up note should cover: (1) exact <code>payload_properties.txt</code> header contract and <code>ZIP_STORED</code> offset/length extraction for streaming; (2) <code>update_engine</code> AIDL/Binder surface on Android 15 (<code>IUpdateEngine</code>); (3) Virtual A/B merge-resume failure handling; (4) signing key management (<code>--package_key</code>, <code>AVB/vbmeta</code>) for the Helix build pipeline.

Table of Contents

1. [Scope & Question](#)
 2. [Executive Summary](#)
 3. [Native Android A/B \(Seamless\) Updates](#)
 4. [Virtual A/B](#)
 5. [Payload Generation: ota_from_target_files](#)
 6. [The Client API: UpdateEngine.applyPayload](#)
 7. [What the Server Side MUST Provide](#)
 8. [Why Native-Only-On-Device May Suffice](#)
 9. [Implications for Helix OTA](#)
 10. [Open Questions / UNVERIFIED Items](#)
 11. [Sources Consulted](#)
 12. [Confidence](#)
-

1. Scope & Question

Helix OTA targets **Android 15 first**, using **native A/B + Virtual A/B** on device with a **custom Go control plane** as the server. This note answers:

- How do Android native A/B and Virtual A/B updates actually work on device?
- How is the update artifact (`payload.bin`) generated (`ota_from_target_files`)?
- What is the on-device client API (`applyPayload`) and its contract?
- What exactly must the **server** provide?
- Why is relying on the **native, on-device** `update_engine` (rather than re-implementing the apply path) likely sufficient for the Android client?

This is a build-vs-wrap decision input. The relevant claim to validate: for Android, Helix should **wrap** the native update path, not replace it.

2. Executive Summary

- **update_engine** is the AOSP daemon that performs A/B updates on device. It reads the active slot, writes the inactive slot, drives the `boot_control` HAL, runs post-install, and handles rollback. It has been the standard A/B mechanism since Android 7/8. (AOSP A/B docs.)
- **Virtual A/B** (default for devices launching Android 11+ per GMS requirement — see UNVERIFIED note) replaces physically duplicated dynamic partitions with **snapshots** written to a Copy-on-Write (COW) area, merged into the base after a confirmed-good boot. It is driven by the same `update_engine` plus userspace components `dm-user` and `snapperd`. This is the mechanism a modern Android 15 device uses.
- The update artifact is **payload.bin** (plus **payload_properties.txt**), generated by `ota_from_target_files` (in `build/make/tools/releasetools`) from the build's **target-files.zip**. Full payloads contain

everything; **incremental/delta** payloads are diffs against a PREVIOUS-target-files.zip.

- The on-device client API is **UpdateEngine.applyPayload(...)** (Java/AIDL, exercised by the AOSP **SystemUpdaterSample**). **applyPayload** is **overloaded**: one form takes a **URL** (HTTP/HTTPS for streaming, or file://), the other takes a **file descriptor**. The client also calls **bind()** to register callbacks (**onStatusUpdate**, **onPayloadApplicationComplete**).
 - **The server is “dumb”**: it must build the OTA package, host **payload.bin** (and metadata), and serve it over HTTPS. For **streaming**, ZIP entries must be stored **uncompressed (ZIP_STORED)** so **update_engine** can range-fetch **payload.bin** by offset/length, and the client must be told the payload’s offset/size plus the hashes/sizes from **payload_properties.txt**. There is **no Google-mandated OTA server protocol** — the orchestration (which device gets which build, rollout %, rollback policy) is entirely the integrator’s responsibility. This is precisely the gap Helix’s Go control plane fills.
 - **Conclusion**: native-on-device **update_engine** does the dangerous part (atomic write, verify, slot switch, rollback) and is battle-tested. Helix’s Android client should be a **thin wrapper** around **applyPayload**; Helix’s value-add is the **build pipeline + serving + rollout/telemetry control plane**, not the apply path.
-

3. Native Android A/B (Seamless) Updates

Source: [Android A/B System Updates](#).

Slots. A/B-capable systems keep two copies (“slots”) A and B of each updatable partition (e.g., boot, system, vendor). The device runs from the **active** slot; the **inactive** slot is the update target and the fallback. Each slot carries bootloader-managed attributes (e.g., bootable / successful).

update_engine role. The **update_engine** daemon: - Reads from current-slot partitions, writes to the unused-slot partitions per the OTA instructions in the payload. - Calls the **boot_control HAL** in a predefined workflow to manage which slot the bootloader boots and to mark slots bootable/successful/unbootable. - After writing all partitions, mounts the new partition and runs **post-install** programs from the new slot. (Constraint per docs: the old kernel must be able to mount the new filesystem and understand the new post-install program format.)

Slot marking & rollback. - Before applying, the current slot is marked successful (so the bootloader won’t fall back), and the target slot is marked unbootable while it is being written. - On success, the previously-unused slot becomes active for next boot. - If the new slot fails to boot, the bootloader **automatically reverts** to the previous slot. This is the core safety property.

Post-reboot verification. After reboot into the new slot, **update_verifier** triggers **dm-verity** integrity checks before **zygote** starts, enabling safe rollback if corruption is detected.

Net: the entire “make it atomic, verify, and revert on failure” responsibility lives in well-tested on-device AOSP components, not in any server.

4. Virtual A/B

Source: [Virtual A/B](#).

Motivation. Classic A/B doubles storage for all duplicated partitions. Virtual A/B keeps a single base for dynamic partitions and writes the **delta to a snapshot**, then **merges into the base partition after confirming a successful boot**. This recovers most of the storage that classic A/B duplicated while keeping the seamless-update UX and rollback safety.

Userspace components. - **dm-user**: a kernel module that lets userspace implement a block device by routing I/O to a daemon. - **snapperd**: the daemon implementing the Android-specific **COW (Copy-on-Write)** format — compression, metadata parsing, and block unpacking.

COW format & compression (Android 12+). The Android COW format supports Copy, Replace (gzip), Zero, and XOR operations. - Reported snapshot-size reduction with compression: ~45% (full OTA) and ~55% (incremental OTA). **UNVERIFIED — needs confirmation against current docs revision; figures originate from the AOSP Virtual A/B page but the exact numbers may differ by Android version.** - **XOR compression** (since Android 13) stores compressed byte-differences instead of full blocks; reported additional 25–40% reduction on devices like Pixel. **UNVERIFIED — same caveat.**

Merge phase. The merge happens **after** a confirmed-good reboot, runs in userspace, and **can be interrupted and resumed**. This matters for Helix telemetry/rollback: a device may be in a post-reboot “merging” state distinct from “fully applied.”

Version requirement. Per the AOSP page, “Virtual A/B is a GMS requirement on devices launching with Android 11 and higher.” **UNVERIFIED for the exact Android-15 wording — confirm against the Android 15/16 docs revision.** Practical takeaway: an Android 15 device shipped through normal channels almost certainly uses Virtual A/B, and is driven by the same `update_engine` + `applyPayload` surface as classic A/B.

5. Payload Generation: `ota_from_target_files`

Sources: [Build OTA packages](#); `ota_from_target_files.py` ([aosp-mirror](#)).

Tool & input. `ota_from_target_files` lives in `build/make/tools/releasetools`. Its primary input is the **target-files.zip** produced by the Android build system (contains partition images + metadata). For deltas it also takes a **PREVIOUS-target-files.zip**.

Full vs incremental. - **Full payload**: generated solely from the target image; contains everything needed to write the inactive slot. Large. - Example: `ota_from_target_files dist_output/tardis-target_files.zip`
`ota_update.zip` - **Incremental / delta payload**: differential update comparing source vs target images; much smaller. - Example: `ota_from_target_files -i PREVIOUS-tardis-target_files.zip dist_output/tardis-target_files.zip incremental_ota_update.zip`

Key flags observed in docs. - `-i` — incremental mode (previous target-files). - `--block` — block-based OTA (Android 8.0+ non-A/B path; A/B is block-based inherently). - `--boot_variable_file <path>` — runtime variable values for dynamic fingerprints (Android 11+). - Multi-SKU (Android 11+): a single package can target multiple SKUs via dynamic fingerprints, e.g. `pre-device=tardis|tardispro`. - Signing: package signing is done with a release key (`--package_key / associated flags`). **UNVERIFIED exact flag spelling on the Android 15 docs revision — confirm; the underlying script scripts/brillo_update_payload and AVB/vbmeta signing are involved.**

Package contents relevant to A/B. - **payload.bin** — the opaque binary blob that `update_engine` consumes; it takes up the majority of the OTA package. - **payload_properties.txt** — companion metadata file holding `FILE_HASH`, `FILE_SIZE`, `METADATA_HASH`, `METADATA_SIZE` (used to verify the payload and required as headers when streaming). Example values (from a real package, illustrative): `FILE_SIZE=379074366`, `METADATA_SIZE=46866`, with base64 hashes. (Sourced from AOSP `payload_generator/payload_properties.cc` and community examples — see Sources.) - **META-INF/com/android/metadata** — preconditions/postconditions: `pre-device`, `pre-build`, `pre-build-incremental`, `post-build`, `post-build-incremental` (derived from `ro.product.device`, `ro.build.fingerprint`, `ro.build.version.incremental`). These let the device/server gate which devices an incremental applies to.

Underlying generator. `ota_from_target_files` ultimately invokes the `update_engine` payload generator (the `brillo_update_payload` script wraps `delta_generator`) to produce and sign `payload.bin`. **The Helix build pipeline must run this tool with the right keys and inputs; it is the producer side of the contract.**

6. The Client API: `UpdateEngine.applyPayload`

Sources: [SystemUpdaterSample README](#); AOSP `update_engine`.

UpdateEngine (Java/AIDL). The Android framework exposes `android.os.UpdateEngine`, a thin client that talks to the `update_engine` daemon over Binder/AIDL. The AOSP **SystemUpdaterSample** is the reference client for non-GMS / custom OTA flows.

applyPayload is overloaded (per AOSP AIDL): 1. **URL form** — `applyPayload(String url, long offset, long size, String[] keyValuePairHeaders)`: starts an update by downloading/streaming the payload from `payload_url` (HTTP/HTTPS for streaming, or `file://` for local). `offset/size` point at `payload.bin` inside the (uncompressed) ZIP; headers carry payload metadata. 2. **File-descriptor form** — `applyPayload(ParcelFileDescriptor fd, long offset, long size, String[] headerKeyValuePairs)`: applies from an already-open FD.

When a client calls `applyPayload`, `update_engine` builds an **InstallPlan** (internal data structure with the update configuration shared across the internal “actions”) and executes the apply.

Callbacks via `bind()`. - `UpdateEngine#bind(callback)` registers callbacks. On successful init, `update_engine` is guaranteed to invoke `onStatusUpdate`. - `onStatusUpdate(int status, float percent)` — called whenever status or progress changes; always fired on bind (unless the daemon fails to init). - `onPayloadApplicationComplete(int errorCode)` — called when an attempt completes or fails.

State management. The sample maintains its own `UpdaterState` because the app can unbind/rebind while an update runs; on rebind it reconciles and may re-issue `applyPayload`. **A Helix Android client must do the same** (survive process death, reconcile daemon state).

Streaming requirement. For streaming, “only necessary ZIP entries are downloaded rather than the entire package,” and `payload.bin` is streamed directly by `update_engine`. This requires the package’s ZIP entries to be **ZIP_STORED** (uncompressed) so byte-range fetches by offset/length are valid.

7. What the Server Side MUST Provide

There is **no Google-defined OTA server protocol**. The device side (`update_engine`) only needs a payload + metadata; everything else is the integrator’s design. Minimum server obligations:

1. **Build & sign the OTA package** (`payload.bin` + `payload_properties.txt` + metadata) via `ota_from_target_files` with the device’s release/signing keys. (Producer side; can be a CI step rather than the live server.)
2. **Host the artifact over HTTPS.** For streaming A/B, store ZIP entries **uncompressed (ZIP_STORED)** and support **HTTP range requests** so `update_engine` can fetch `payload.bin` by offset/length.
3. **Provide the apply parameters to the client:**
 - `payload_url`
 - offset and size of `payload.bin` within the package
 - Header key/values from `payload_properties.txt`: `FILE_HASH`, `FILE_SIZE`, `METADATA_HASH`, `METADATA_SIZE` (plus optionally `SWITCH_SLOT_ON_REBOOT`, `RUN_POST_INSTALL` — **UNVERIFIED exact optional-header set; confirm in `payload_properties.cc` / AIDL docs**).
4. **Targeting / preconditions.** Decide which device (by pre-device, pre-build, fingerprint) is eligible for a given full/incremental payload — incrementals only apply to the exact source build. The server matches device-reported build info to the right payload.
5. **Orchestration (Helix value-add, not provided by AOSP):** staged/percentage rollout, pause/resume, A/B (experiment) cohorts, rollback policy/recall, telemetry ingestion, signing-key custody, device inventory. **None of this exists in AOSP** — `update_engine` just applies whatever it is handed.

The division is clean: **AOSP owns “apply safely on device”; the server owns “decide, deliver, observe.”**

8. Why Native-Only-On-Device May Suffice

For the **Android client**, re-implementing the apply path would be high-risk and low-value. Native `update_engine` already provides, on device:

- **Atomicity** — inactive-slot writes; the running system is never mutated in place.
- **Verification** — payload hashes (`FILE_HASH/METADATA_HASH`), AVB/dm-verity, `update_verifier` before zygote.
- **Automatic rollback** — bootloader reverts to the prior slot if the new slot fails to boot/verify.
- **Storage efficiency** — Virtual A/B snapshots + COW compression.
- **Streaming** — range-fetch of `payload.bin`, partial downloads.
- **Resumability** — interrupted/resumable apply and Virtual A/B merge.
- **A stable, supported client API** — `UpdateEngine.applyPayload` + callbacks, used by the AOSP reference sample.

These are exactly the “zero corruption, atomic, verified, revertible” guarantees in the Helix mission. Reproducing them outside AOSP would mean re-deriving years of hardened boot/verify/rollback logic and tracking it across Android releases — a maintenance liability with no upside on Android.

Therefore for Android, Helix should wrap, not replace: a thin Android client that (a) talks to the Helix Go control plane to learn which payload to apply, (b) calls `applyPayload` with the URL/offset/size/headers, (c) relays `onStatusUpdate` / `onPayloadApplicationComplete` as telemetry, and (d) reconciles state across rebinds. The “native-only-on-device” path is sufficient because the on-device daemon already covers the safety-critical surface; Helix’s job is upstream (build/serve) and orchestration (rollout/telemetry/rollback policy).

Caveat / boundary: this conclusion is Android-specific. Helix’s universal ambition (Linux/Windows later) cannot rely on `update_engine`; those need different backends (RAUC/SWUpdate/OSTree/Mender-style). So “native-only” is the right Android backend choice, behind the same Helix client abstraction — not a universal mechanism.

9. Implications for Helix OTA

- **Android backend = wrap `update_engine`.** Confirms the design intent of an Android client SDK that is a thin wrapper over `applyPayload`. Do not build a custom Android apply engine.
- **Build pipeline must run `ota_from_target_files`.** Helix needs reproducible CI that consumes `target-files.zip`, produces full + incremental `payload.bin`, and signs with managed keys. Signing-key custody is a first-class concern.
- **Serving must be streaming-correct.** OTA ZIPs served as `ZIP_STORED`; HTTPS with range support; the control plane returns `{url, offset, size, FILE_HASH, FILE_SIZE, METADATA_HASH, METADATA_SIZE}` to the device.
- **Control plane owns rollout/telemetry/rollback** — the part AOSP deliberately leaves open. This is where the Go server’s differentiation lives (staged %, cohorts, recall, inventory, telemetry from `onStatusUpdate/merge` state).
- **Track Virtual A/B states** — “applied, pending reboot, merging, merged” must be modeled in telemetry; merge can be interrupted/resumed.

10. Open Questions / UNVERIFIED Items

1. **Exact applyPayload AIDL signature & optional headers on Android 15** (e.g., SWITCH_SLOT_ON_REBOOT, RUN_POST_INSTALL, DISABLE_DOWNLOAD_RESUME) — confirm against the Android 15 IUpdateEngine/payload_properties.cc.
 2. **Virtual A/B GMS version threshold wording for Android 15/16** — the “Android 11+” statement is from the AOSP page; re-verify current phrasing. UNVERIFIED.
 3. **Compression % figures (45%/55%/25–40%)** — from the AOSP Virtual A/B page; treat as approximate and version-dependent. UNVERIFIED.
 4. **Signing flags** — exact ota_from_target_files signing flags (--package_key, AVB/avbtool, vbmeta) for the Helix pipeline. UNVERIFIED spelling.
 5. **ZIP_STORED + range serving** details and how the control plane computes payload.bin offset/size — to be specified in the serving note.
 6. **OEM/board specifics (Orange Pi 5 Max / RK3588)** — boot_control HAL availability and whether the target build ships A/B or Virtual A/B; needs hands-on confirmation on the actual board.
-

11. Sources Consulted

Primary (AOSP, official): - Android A/B System Updates — <https://source.android.com/docs/core/ota/ab> - Virtual A/B — https://source.android.com/docs/core/ota/virtual_ab - Build OTA packages (ota_from_target_files) — <https://source.android.com/docs/core/ota/tools> - SystemUpdaterSample README (UpdateEngine.applyPayload, bind, callbacks) — https://android.googlesource.com/platform/bootable/recovery/+/main/updater_sample/README.md - update_engine (Git) — https://chromium.googlesource.com/aosp/platform/system/update_engine/ - payload_generator/payload_properties.cc — https://android.googlesource.com/platform/system/update_engine/+/refs/tags/android-s-preview-1/payload_generator/payload_properties.cc - payload_consumer/payload_metadata.cc — https://android.googlesource.com/platform/system/update_engine/+/master/payload_consumer/payload_metadata.cc - ota_from_target_files.py (aosp-mirror) — https://github.com/aosp-mirror/platform_build/blob/master/tools/releasetools/ota_from_target_files.py - scripts/brillo_update_payload — https://android.googlesource.com/platform/system/update_engine/+/refs/heads/oreo-m4-s2-release/scripts/brillo_update_payload

Secondary / community (used only for concrete example values; flagged accordingly): - Example payload_properties.txt values & update_engine_client --headers usage — NXP Community thread (<https://community.nxp.com/t5/i-MX-Processors/AOSP-Update/td-p/1006438>) and a real crosshatch OTA log gist (<https://gist.github.com/nooriro/313201c9687a409b2db0f32dead05d2a>).

No statistics, dates, or citations were fabricated. Items not directly verified against the current docs revision are tagged “UNVERIFIED — needs confirmation.”

12. Confidence

Overall confidence: HIGH on the architecture and the build-vs-wrap conclusion (these are well-documented, stable AOSP mechanisms confirmed across multiple primary sources). **MEDIUM** on the exact numeric/version constants and the precise Android 15 AIDL header set — flagged UNVERIFIED above and slated for the continuation note.